

MINISTRY OF EDUCATION AND RESEARCH



TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Assignment 1

Software Design

Author: Maria Moruțan

Group: 30433

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

March 2026

Contents

1	Problem definition	2
2	Analysis phase - Use cases & requirements	2
2.1	Functional requirements	2
2.2	Use cases	3
3	Design phase	4
3.1	Architecture overview	4
3.2	Entity-relationship diagram	4
3.3	Component: Backend Service Layer	4
3.4	Design patterns	5
3.5	Class diagrams	5
3.6	Sequence diagrams	6
4	Implementation phase - Application	7
4.1	Role-Based Access Control Logic	7
4.2	Search and Sorting Implementation	7
4.3	User interface design	7
5	Testing and validation	8

1 Problem definition

Implement an application that manages a set of domain entities (for a pet shop). The application must support multiple user roles and allow users to interact with the system according to their permissions. The purpose of this assignment is to implement a simple but well-structured application using object-oriented principles and a layered architecture. The Pet Management System allows users to manage information about pets and related items in a shop or environment. The system supports different types of users with varying permissions: Visitors, who can only view public information; Administrators, who have full access to all functionalities; and Sellers, a secondary role who can add but not update pets or delete them.

The application is structured in three main phases:

- Analysis phase—identifying the functional requirements and use cases of the system.
- Design phase—defining the architecture, database structure, components, and design patterns.
- Implementation phase—building the actual application with the chosen technologies and providing a user interface for interaction.

The deliverables for this assignment include:

- Fully functional Pet Management application with MVC architecture.
- Role-based access control implemented for Visitors, Administrators, and Sellers.
- Documentation including analysis, design, implementation, and testing phases.
- Screenshots and explanations of the user interface.
- Source code organized according to MVC best practices.

2 Analysis phase - Use cases & requirements

2.1 Functional requirements

The functional requirements of the Pet Management System include:

- User authentication and role-based access control.
- CRUD operations on pets (Create, Read, Update, Delete) with permissions restricted by role.
- CRUD operations on items (toys, accessories, food) with permissions restricted by role.
- Searching and filtering pets by name.
- Sorting pets by name, age, or price in ascending or descending order.
- Displaying pets and items in a user-friendly interface with role-specific functionalities.

2.2 Use cases

Below are the main use cases of the system:

- **Visitor views pets and items:** Visitor accesses the pet/items list. The system displays all available pets/items. Visitor cannot modify data.
- **Administrator manages pets and items:** Admin logs in, creates, edits, or deletes pets/items. The system updates the database accordingly.
- **Seller adds pets and items:** Seller logs in, creates new pets/items. Seller cannot update or delete pets or items.
- **Search and sort pets:** Any user can filter pets by name and sort them by name, age, or price. The system displays the results accordingly.

Use case diagram:

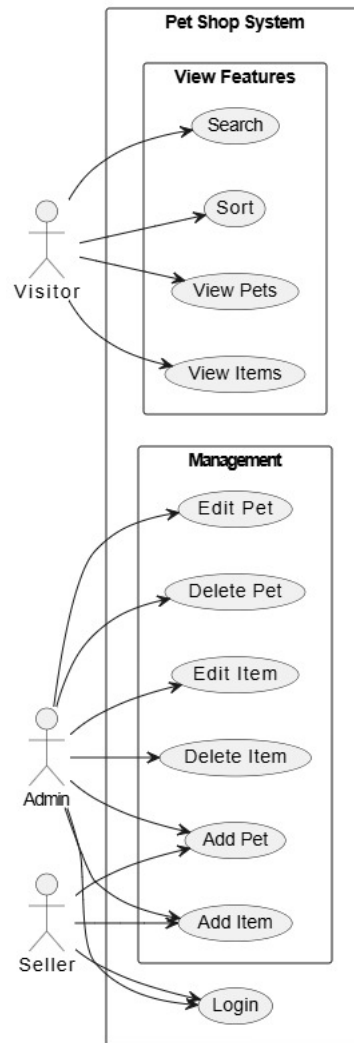


Figure 1: Use case diagram showing interactions between Visitors, Administrators, and Sellers with the system

3 Design phase

3.1 Architecture overview

The application follows a layered MVC architecture, which provides a clean separation between the data, the user interface, and the application logic.

- **Model:** This layer is subdivided into Entities (representing database tables), Repositories (handling the abstraction of SQL queries via Spring Data JPA), and Services (where the business rules, such as role-validation and data-filtering, reside).
- **View:** Built using Thymeleaf, a server-side Java template engine. It allows for the dynamic rendering of HTML based on the user's session attributes.
- **Controller:** It intercepts HTTP requests, interacts with the Service layer, and directs the user to the appropriate View.

3.2 Entity-relationship diagram

The data model is designed around a relational schema. A key feature is the Many-to-One relationship between `Item` and `Pet`, allowing specific products to be marketed for individual animals.

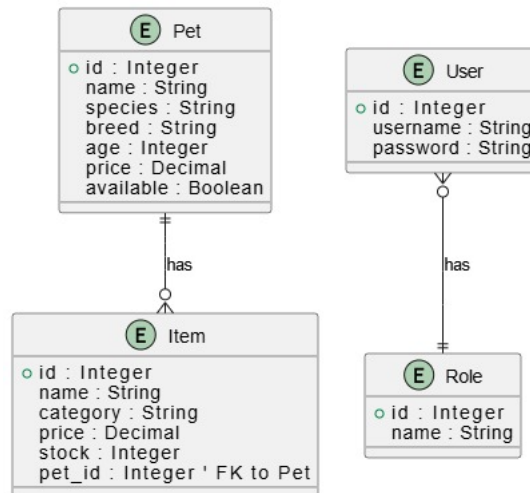


Figure 2: Entity-relationship diagram for the Pet Management System

3.3 Component: Backend Service Layer

The Backend Service acts as an intermediary. By implementing a Service layer, we ensure that the Controllers remain with not as many responsibilities. For example, when a `Pet` is deleted, the Service layer can handle the logic of unlinking associated `Items` or checking if the current user has the `ADMIN` role before the Repository executes the delete command.

3.4 Design patterns

- **MVC Pattern:** Essential for web scalability and separating frontend concerns from back-end logic.
- **Repository Pattern:** By using Spring Data JPA, we decouple the application from the specific database dialect, allowing for easy migration between H2 and PostgreSQL.
- **Dependency Injection:** Facilitated by the Spring Framework, this allows for better testability by injecting mock services into controllers during the testing phase.

3.5 Class diagrams

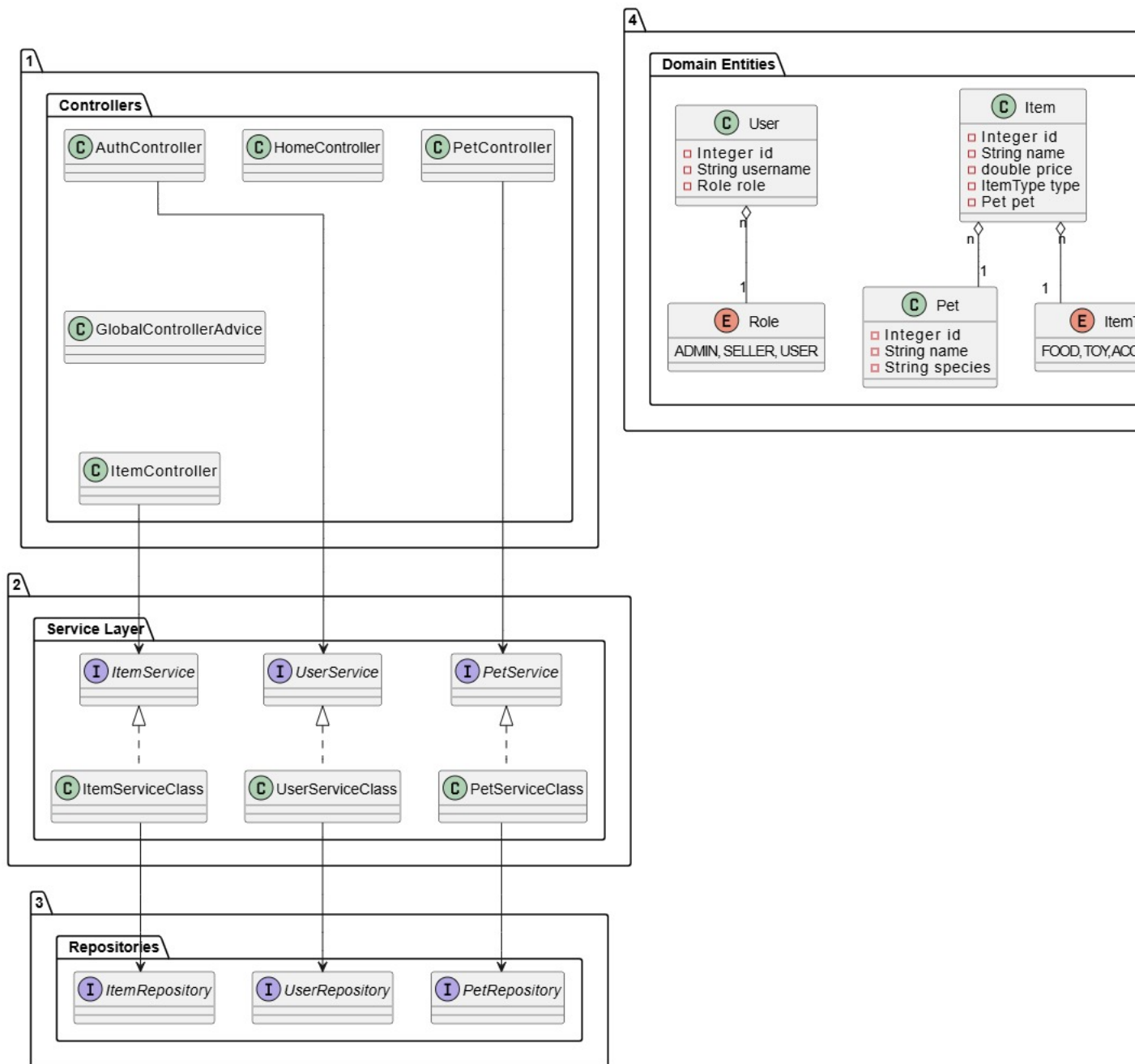


Figure 3: Class diagram showing Pet, User, Role, Services, and Repositories

3.6 Sequence diagrams

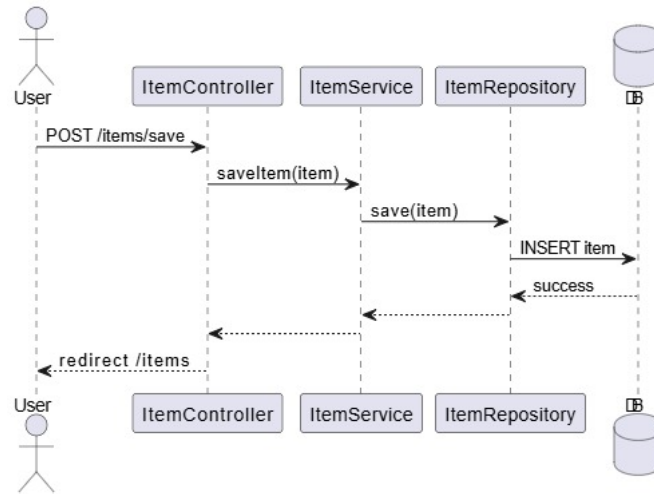


Figure 4: Sequence diagram for adding a new Item by an Administrator

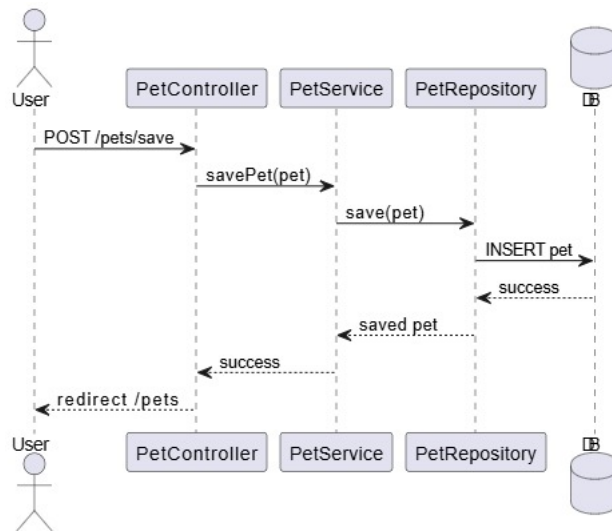


Figure 5: Sequence diagram for adding a new Pet by an Administrator

4 Implementation phase - Application

4.1 Role-Based Access Control Logic

A core requirement of the system is the implementation of RBAC. Instead of hardcoding permissions into every HTML page, the system utilizes the `HttpSession` to track the `User` object.

- **Admins** have access to all POST/PUT/DELETE mappings.
- **Sellers** are granted access to `/add` mappings but are restricted from `/edit` and `/delete` via Controller-level checks.
- **Visitors** are restricted to GET requests for viewing lists and search results.

4.2 Search and Sorting Implementation

To enhance user experience, the system implements dynamic sorting. This is achieved by passing sort parameters (e.g., `?sort=price&dir=asc`) from the Thymeleaf view to the Service layer, which then utilizes the `Sort` object in JPA Repository methods to return ordered data without writing custom SQL.

4.3 User interface design

The UI was developed with a "Mobile First" approach using Bootstrap 5. Tables are wrapped in responsive containers, and Font Awesome icons are used to provide intuitive visual cues for actions like "Edit" (pencil icon) and "Delete" (trash icon).

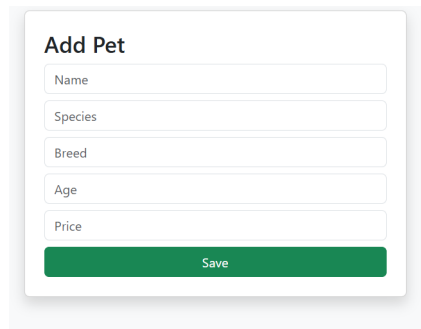
Pets List

[Add New Pet](#)

Search by name... [Search](#)

Name  	Species	Breed	Age  	Price  	Actions
Orange	Cat	Maincoon	3	400.00	
Pookie	Dog	Doberman	4	200.00	

Figure 6: Seller dashboard demonstrating restricted action visibility.



The image shows a web form titled "Add Pet". It contains five input fields stacked vertically: "Name", "Species", "Breed", "Age", and "Price". Each field is a simple text box with a light gray border. Below the "Price" field is a solid green button with the word "Save" in white text. The entire form is centered on a light gray background.

Figure 7: Pet registration form with server-side validation.

5 Testing and validation

The application was tested using:

- Unit tests for service methods to ensure CRUD, search, and sorting logic works correctly.
- Manual testing of the user interface to verify role-based access, search, sorting, and CRUD operations.
- Validation of form inputs to ensure correct data types and required fields.

Testing confirmed that the application meets the functional requirements defined in the analysis phase.